

Chapter 2: Classes & Objects

- **Text Book : Java How to Program**

- Deitel & Deitel, 9th Edition

- All the program numbers in the presentation are given from Deitel & Deitel for your reference.

- **Reference Book : Java Complete Reference**

Classes, Objects & Methods

- A class is a user defined data type that serves as a template to define its properties.
- Once a class type has been defined we can create we can create instances of classes, which are actual objects.

- BASIC FORM

```
public class classname [extends superclassname]
{
    [fields declaration;]
    [method declaration;]
}
```

Field Declaration

- Data is encapsulated in class by placing the data fields inside the body of class definition.
- These variables are called as **instance variables** since they are created whenever object of class is instantiated.

```
public class Rectangle  
{  
    private int length;  
    private int width;  
}
```

Can also be declared as : int length, width;

Instance variables : member variables, data members

Method Declaration

- General form:

```
Return_type method_name (parameter_list)
{
    method _body;
}
```

Parts of method declaration:

1. Name of method
2. Return type (type of value method returns)
3. List of parameters
4. Body of method

Example

```
public class Rectangle
{
    int length;
    int width;

    void setData (int x, int y)
    {
        length = x;
        width = y;
    }
}
```

Example

```
public class Rectangle
{
    private int length;
    private int width;

    public void setData(int x, int y)
    {
        length = x;
        width = y;
    }

    public int rectArea ()
    {
        int area = length * width;
        return (area);
    }
}
```

Creating Objects

- An object in JAVA is a block of memory that contains space to store all the instance variables.
- Objects in Java are created using new operator.
- The new operator creates object of specified class and returns reference to that object.

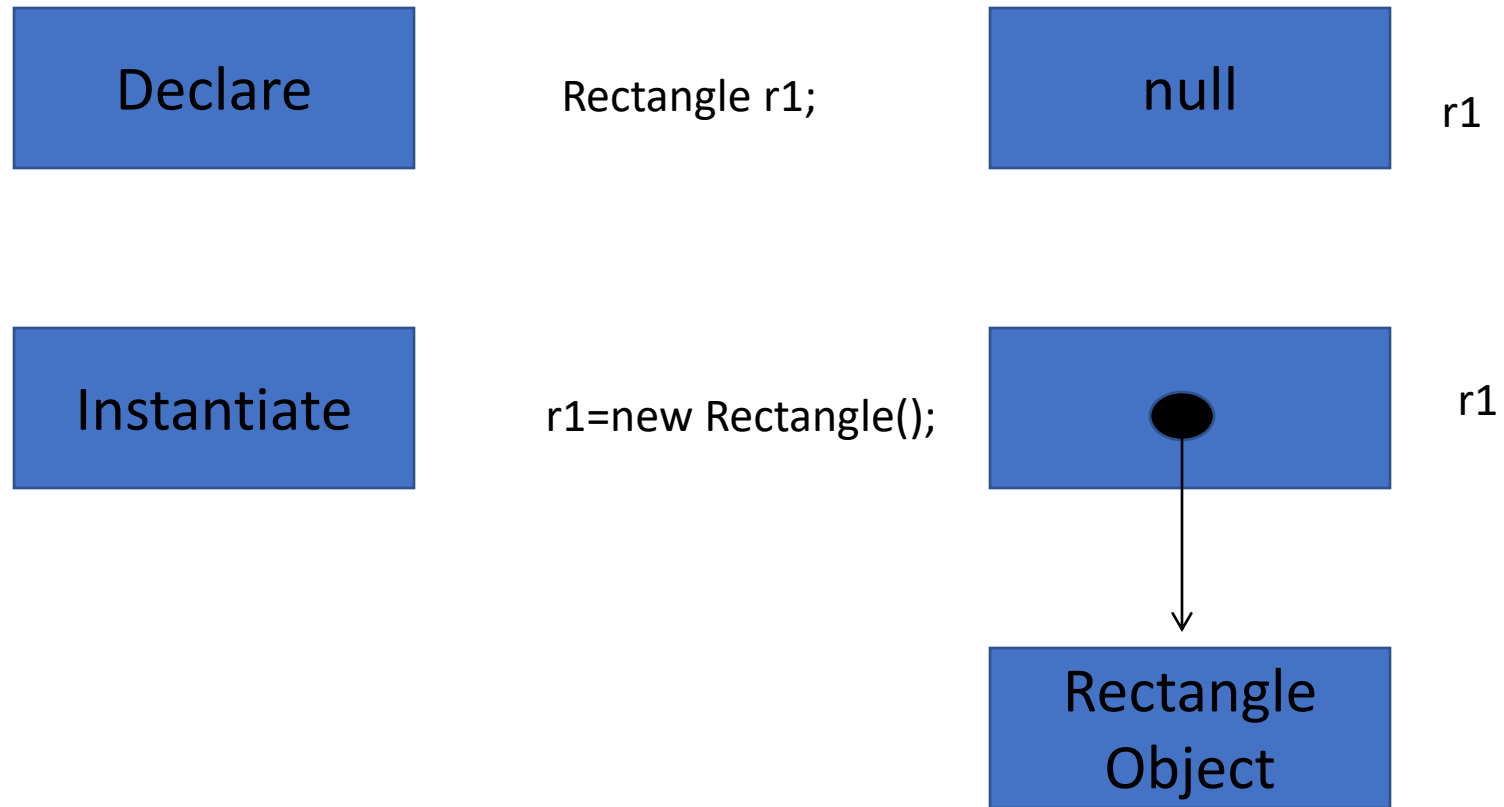
Rectangle r1; // declaring object

r1 = new Rectangle (); // instantiating object

The first statement declares variable to hold the object reference and second one actually assigns the object reference to the variable.

Rectangle r1 = new Rectangle();

Creating Objects



Accessing Class Members

- The instance variables and methods of a class cannot be accessed directly outside the class.
- Concerned object and dot operator is required.
Objectname.variablename = value;
Objectname.methodname(parameterlist);

Example

```
public Class RectangleAccess
{
    public static void main(String args[])
    {
        int area1, area2;
        Rectangle r1 = new Rectangle();
        Rectangle r2 = new Rectangle ();
        r1.length = 10; //error
        r1.width=12; //error
        area1 = r1.length * r1.width; //error
        r1.setData (5, 3);
        area1 = r1.rectArea();
        r2.setData(20, 10);
        area2 = r2.rectArea();
        System.out.println("Area1 = " + area1);
        System.out.println("Area2 = " + area2);
    }
}
```

Output:

Area1 = 120 (Compile both the classes: Rectangle and RectangeAccess)

Area2 = 200 (Execute/Run: RectangleAccess)

GradeBook Class

```
1 // Fig. 3.1: GradeBook.java
2 // Class declaration with one method.
3
4 public class GradeBook
5 {
6     // display a welcome message to the GradeBook user
7     public void displayMessage()
8     {
9         System.out.println( "Welcome to the Grade Book!" );
10    } // end method displayMessage
11 } // end class GradeBook
```

Performs the task of displaying a message on the screen; method `displayMessage` must be called to perform this task

Fig. 3.1 | Class declaration with one method.

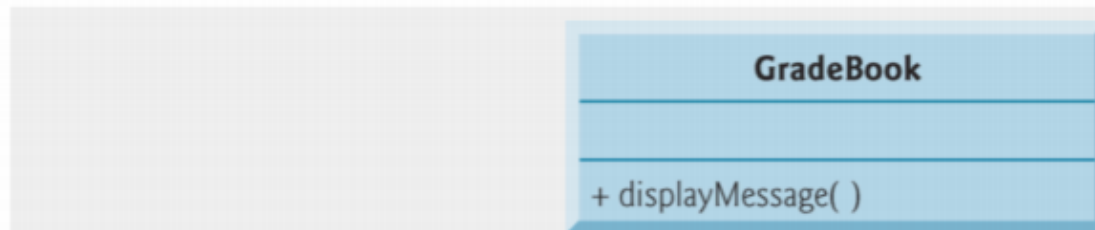


Fig. 3.3 | UML class diagram indicating that class `GradeBook` has a `public displayMessage` operation.

GradeBook Class

- GradeBook class
 - contains – Method `displayMessage()`
 - Class declaration starts with “public class” followed by name of class
 - **public class GradeBook{**
 - The class contains method `displayMessage()`
 - Method declaration starts with keyword “public” that tells it is available to the public (outside the class) followed by return type (void) and name of method (`displayMessage()`)
 - The method has a print statement that prints “Welcome to Grade Book !”
- UML diagram of GradeBook Class is shown in previous slide:
 - In UML each class is modelled in class diagram which has 3 compartments:
 - Compartment 1 : Name of Class
 - Compartment 2: Class Attributes (Instance Variables)
 - Compartment 3: Class Operations (Methods in Java)

GradeBookTest Class (Figure 3.2)

```
1  // Fig. 3.2: GradeBookTest.java
2  // Creating a GradeBook object and calling its displayMessage method.
3
4  public class GradeBookTest
5  {
6      // main method begins program execution
7      public static void main( String[] args )
8      {
9          // create a GradeBook object and assign it to myGradeBook
10         GradeBook myGradeBook = new GradeBook();
11
12         // call myGradeBook's displayMessage method
13         myGradeBook.displayMessage();
14     } // end main
15 } // end class GradeBookTest
```

Creates a GradeBook object and assigns it to variable myGradeBook

Invokes method displayMessage on the GradeBook object that was assigned to variable myGradeBook

Welcome to the Grade Book!

GradeBookTest Class

- GradeBookTest class is written to create object of GradeBook class
- The GradeBookTest class has main() method.

```
public static void main (String args[])  
{  
    GradeBook myGradeBook = new GradeBook();  
    myGradeBook.displayMessage();  
}
```

- Inside main() method, object of GradeBook class is created

```
GradeBook myGradeBook = new GradeBook();
```

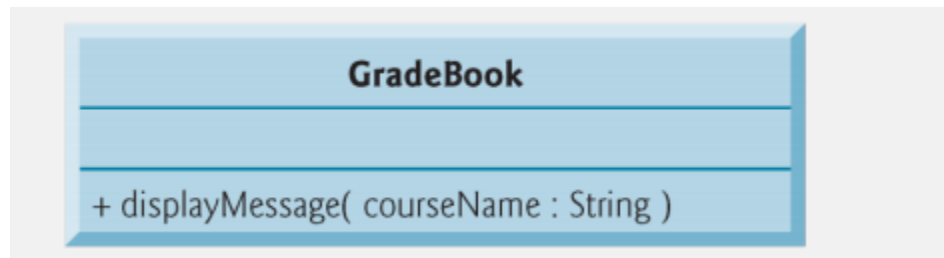
- Then, using myGradeBook object, displayMessage() method is called:
myGradeBook.displayMessage();

Declaring a Method with Parameter (Fig 3.4)

```
1 // Fig. 3.4: GradeBook.java
2 // Class declaration with a method that has a parameter.
3
4 public class GradeBook
5 {
6     // display a welcome message to the GradeBook user
7     public void displayMessage( String courseName )
8     {
9         System.out.printf( "Welcome to the grade book for\n%s!\n",
10             courseName );
11     }
12 } // end class GradeBook
```

Parameter `courseName` provides the additional information that the method requires to perform its task

Parameter `courseName`'s value is displayed as part of the output



UML Class diagram showing `GradeBook` class with method `displayMessage` having parameter `courseName` of type: `String` (Fig 3.6)

Declaring a Method with Parameter (continued..)

- Earlier example of GradeBook had method `displayMessage()` with no parameters.
- The GradeBook class of fig 3.4 has `displayMethod (String courseName)`, that has string parameter “courseName”
 - The method displays message “Welcome to Grade Book for ” followed by `courseName` that is passed as argument while calling the method.
 - When a method with parameters is called, the number of arguments in a method call must match the number of parameters in parameter list of method declaration
 - Also the data types of arguments must match the corresponding parameter types
- UML diagram of GradeBook class with `displayMessage(String courseName)` having parameter is shown

GradeBookTest Class

- To create object of GradeBook class and call DisplayMessage() method, GradeBookTest class is written:

```
1  // Fig. 3.5: GradeBookTest.java
2  // Create GradeBook object and pass a String to
3  // its displayMessage method.
4  import java.util.Scanner; // program uses Scanner
5
6  public class GradeBookTest
7  {
8      // main method begins program execution
9      public static void main( String[] args )
10     {
11         // create Scanner to obtain input from command window
12         Scanner input = new Scanner( System.in );
13
14         // create a GradeBook object and assign it to myGradeBook
15         GradeBook myGradeBook = new GradeBook();
16
17         // prompt for and input course name
18         System.out.println( "Please enter the course name:" );
19         String nameOfCourse = input.nextLine(); // read a line of text
20         System.out.println(); // outputs a blank line
21     }
```

← Reads a String from the user

```
22      // call myGradeBook's displayMessage method
23      // and pass nameOfCourse as an argument
24      myGradeBook.displayMessage( nameOfCourse );
25  } // end main
26 } // end class GradeBookTest
```

Passes the value of nameOfCourse as the argument to method displayMessage

```
Please enter the course name:
CS101 Introduction to Java Programming

Welcome to the grade book for
CS101 Introduction to Java Programming!
```

- The GradeBookTest class has main() method
- The main() method creates object of GradeBook class, myGradeBook
- Gets the name of course from user

```
GradeBook myGradeBook = new GradeBook();

// prompt for and input course name
System.out.println( "Please enter the course name:" );
String nameOfCourse = input.nextLine(); // read a line of text
System.out.println(); // outputs a blank line
```

Reads a String from the user

- Passes the entered course name as parameter to displayMessage(): myGradeBook.displayMessage(nameOfCourse);
- Compile both the classes: GradeBook and GradeBookTest
- Run/Execute GradeBookTest class

Instance Variables, set methods and get methods

- Variables declared in the body of a particular method are called **local variables** and can be used only within that method.
- Variables declared in a class are called as attributes or fields. They are declared inside a class and outside the bodies of methods in the class.
- When object of a class is declared, the variables are declared in memory.
- Every object of the class maintains its own copy of attributes (variables).
- They are also called as instance variables as every object has separate instance of these variables in memory.

GradeBook Class with Instance Variable

Program – modified GradeBook class

- The class GradeBook maintains instance variable `courseName` that can be used or modified at any time.
- The class has three methods:
 - `setCourseName` – set method to set/store the value of `courseName`
 - `getCourseName` – get method to read the value of `courseName`
 - `displayMessage` – with no parameters. Displays welcome message with `courseName` value
- For every instance variable `set()` and `get()` methods can be written
 - `set()` - assigns value to variable (naming – `setNameofVariable()` – `setCourseName()`)
 - Set method has parameter which is the value to be assigned to the variable
 - `get()` –reads the value of variable (naming – `getNameofVariable()` – `getCourseName()`)
 - Get method does not have parameter but has return type. Returns value of variable

```
1 // Fig. 3.7: GradeBook.java
2 // GradeBook class that contains a courseName instance variable
3 // and methods to set and get its value.
```

```
4
5 public class GradeBook
6 {
7     private String courseName; // course name for this GradeBook
8
9     // method to set the course name
10    public void setCourseName( String name )
11    {
12        courseName = name; // store the course name
13    } // end method setCourseName
14
15    // method to retrieve the course name
16    public String getCourseName()
17    {
18        return courseName;
19    } // end method getCourseName
20
21    // display a welcome message to the GradeBook user
22    public void displayMessage()
23    {
24        // calls getCourseName to get the name of
25        // the course this GradeBook represents
26        System.out.printf( "Welcome to the grade book for\n%s!\n",
27                           getCourseName() );
28    } // end method displayMessage
29 } // end class GradeBook
```

Each GradeBook object maintains its own copy of instance variable `courseName`

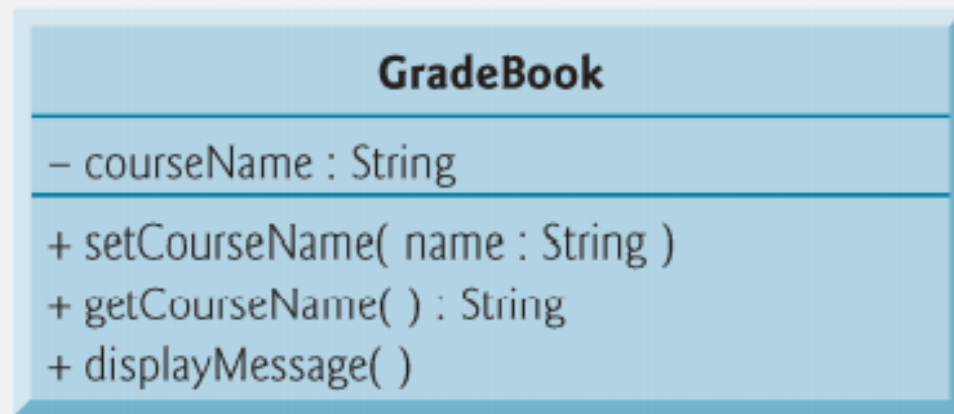
Method allows client code to change the `courseName`

Method allows client code to obtain the `courseName`

No parameter required; all methods in this class already know about instance variable `courseName` and the class's other methods

Good practice to access your instance variables via `set` or `get` methods

UML Diagram of GradeBook



GradeBookTest Class

```
1  // Fig. 3.8: GradeBookTest.java
2  // Creating and manipulating a GradeBook object.
3  import java.util.Scanner; // program uses Scanner
4
5  public class GradeBookTest
6  {
7      // main method begins program execution
8      public static void main( String[] args )
9      {
10         // create Scanner to obtain input from command window
11         Scanner input = new Scanner( System.in );
12
13         // create a GradeBook object and assign it to myGradeBook
14         GradeBook myGradeBook = new GradeBook();
15
16         // display initial value of courseName
17         System.out.printf( "Initial course name is: %s\n\n",
18             myGradeBook.getCourseName() );
19
20         // prompt for and read course name
21         System.out.println( "Please enter the course name:" );
22         String theName = input.nextLine(); // read a line of text
23         myGradeBook.setCourseName( theName ); // set the course name
```

Gets the value of the myGradeBook object's courseName instance variable

Sets the value of the courseName instance variable

```
24      System.out.println(); // outputs a blank line
25
26      // display welcome message after specifying course name
27      myGradeBook.displayMessage();
28  } // end main
29 } // end class GradeBookTest
```

Displays the GradeBook's message, including the value of the `courseName` instance variable

Initial course name is: null

Please enter the course name:

CS101 Introduction to Java Programming

Welcome to the grade book for

CS101 Introduction to Java Programming!

Primitive type and reference type

- Java's types are divided into: Primitive types and reference types
- Primitive types: Boolean, byte, char, short, int, long, float and double
- If a variable is not of primitive type, then it is **reference**.
- Classes that let us the type of object are reference types.
- Primitive type variables:
 - Can stored one value of its type at a time
 - Primitive instance variables are initialized by default
 - Instance variables of type byte, char, short, int, long, float, double are initialized to zero and variables of type Boolean to false
 - You can specify your own initial value at the time of declaration
`private int n = 10;`

Primitive type and reference type

- Variables of Reference types – reference
- Used to store location of objects in computer memory
- Objects that are referenced many contain many instance variables
- Reference type instance variables are initialized by default to “null” – reference to nothing

Initializing Objects with Constructors

- Java supports a special type of method, called constructor.
- Constructor enables an object to initialize itself when it is created.
- Constructors have the same name as that of the class.
- They do not specify return type, not even void.
- This is because they return instance of class itself.
- Constructors are called automatically when object of a class is created.

Default Constructor

- Constructor with no arguments is called default constructor.

```
public class perimeter
{
    private int length;
    private int width;
    perimeter () //Constructor with no arguments: default constructor
    {
        length=0;
        width=0;
        System.out.println("Inside Default Constructor \n Instance Variables initialized");
    }
    public void setData (int x, int y)
    {
        length=x;
        width = y;
    }
    public int cal_perimeter()
    {
        int peri = 2 * (length + width);
        return peri;
    }
}
```

Default Constructor

```
public class PerimeterTest
{
    public static void main(String args[])
    {
        Perimeter p1 = new Perimeter();
        int l, w;
        l = 20;
        w = 5;
        p1.setData (l, w);
        int peri1 = p1.cal_perimeter();
        System.out.println("Perimeter = " + peri1);
    }
}
```

- Output

Inside Default Constructor

Instance Variables initialized

peritemer = 50

Parameterized Constructor

A Constructor with parameters is called parameterized constructor

```
public class Perimeter {  
    private int length;  
    private int width;  
  
    Perimeter (int l, int w) //Parameterized Constructor  
    {  
        length=l;  
        width=w;  
        System.out.println("Inside parameterized  
        Constructor \n Instance Variables initialized");  
    }
```

```
        public void setData (int x, int y)  
        {  
            length=x;  
            width = y;  
        }  
  
        public int cal_perimeter()  
        {  
            int peri = 2 * (length + width);  
            return peri;  
        }  
    } //end of perimeter class
```

Testing Perimeter Class (Parameterized Constructor)

```
public class PerimeterTest {  
  
    public static void main(String[] args) {  
  
        Perimeter p2 = new Perimeter(10,30); //parameterized constructor is called  
        Perimeter p1 = new Perimeter (0,0);  
  
        System.out.println("Perimeter =" + p2.cal_perimeter());  
  
        int l, w;  
        l = 5; w = 2;  
        p1.setData (l,w);  
        System.out.println("Perimeter =" + p1.cal_perimeter());  
    }  
}
```


output

Inside parameterized constructor

Instance variables initialized

Perimeter = 80

Perimeter = 14

Parameterized Constructor – GradeBook Class Example

A Constructor with parameters is called parameterized constructor

```
public class GradeBook {  
  
    private String courseName;  
  
    GradeBook(String name) //parameterized Constructor  
    {  
        courseName = name;  
    }  
  
    public void setCourseName(String name)  
    {  
        courseName = name;  
    }  
}
```

Parameterized Constructor

```
public String getCourseName()
{
    return courseName;
}

public void displayMessage()
{
    System.out.println("Welcome to GradeBook of " + getCourseName());
}

}
```

Parameterized Constructor (Testing GradeBook Class)

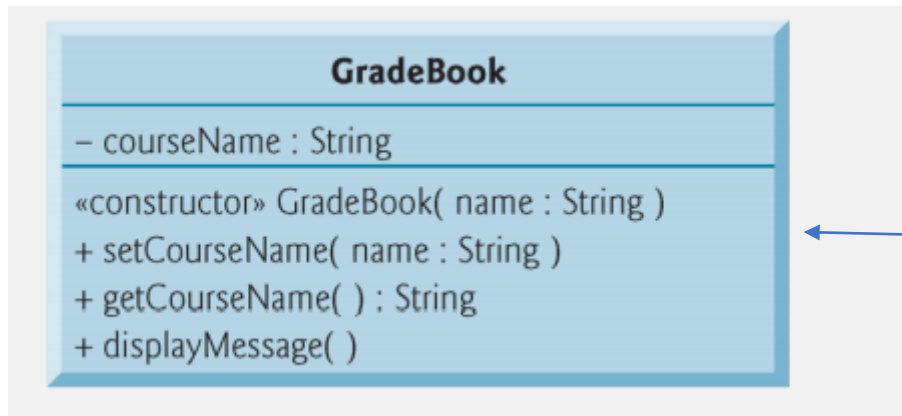
```
public class Example_GradeBook_ParaConst {  
  
    public static void main(String[] args) {  
  
        GradeBook gradeBook1 = new GradeBook("OOP with Java");  
        GradeBook gradeBook2 = new GradeBook("Maths-IV");  
  
        System.out.println("GradeBook1 course name =" + gradeBook1.getCourseName());  
        System.out.println("GradeBook2 course name =" + gradeBook2.getCourseName());  
    }  
}
```

Parameterized Constructor

GradeBook1 course name =OOP with Java

GradeBook2 course name =Maths-IV

Output



UML Diagram

Constructor Overloading

- A class can have more than one constructors. This is called as constructor overloading
- When there are multiple constructors in a class, they should have different number and/or type of parameters
- Which version of constructor will be called depends on the number and/type or parameters passed.

```
public class Perimeter {  
    private int length;  
    private int width;  
  
    Perimeter () //Default Contructor  
    {  
        length = 0;  
        width = 0;  
        System.out.println("Inside Default Constructor \n  
        Instance Variables initialized");  
    }  
  
    Perimeter (int l, int w) //Parameterized Constructor  
    {  
        length=l;  
        width=w;  
        System.out.println("Inside parameterized  
        Constructor \n Instance Variables  initialized");  
    }  
}
```

```
        public void setData (int x, int y)  
        {  
            length=x;  
            width = y;  
        }  
  
        public int cal_perimeter()  
        {  
            int peri = 2 * (length + width);  
            return peri;  
        }  
  
    } //Perimeter Class Ends
```

Testing Perimeter Class (Constructor Overloading)

```
public class Example_Perimeter_DefaultConst {  
  
    public static void main(String[] args) {  
  
        Perimeter p1 = new Perimeter(); //Default constructor called lenght=0, width=0  
        p1.setData(10,20);  
        System.out.println("Perimeter =" + p1.cal_perimeter());  
  
        Perimeter p2 = new Perimeter(10,30); //parameterized constructor is called  
        System.out.println("Perimeter =" + p2.cal_perimeter());  
  
    }  
}
```

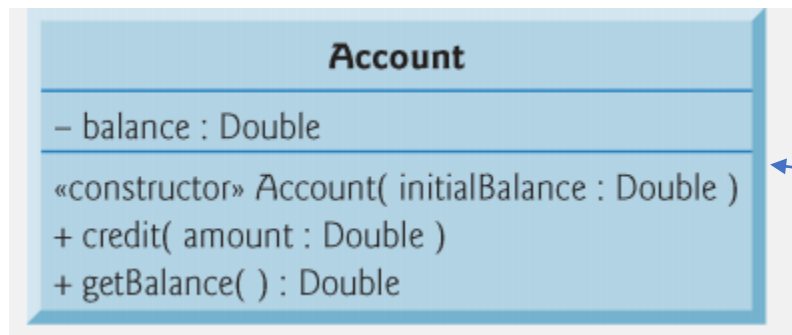

Floating Point Numbers & Type Double

- Java Provides two primitive types for storing floating point numbers in memory: **float and double**
- Variables of type float : represent **single precision floating point numbers and can represent upto seven significant digits**
- Variables of type double: represent **double precision floating point numbers and provide 15 significant points**
- Variables of type double require twice as much memory as that of float

Accounts Class Example

```
public class Account {  
  
    private double balance;  
  
    public Account() // default constructor  
    {  
        balance = 0.0;  
        System.out.println ("Default constructor");  
    }  
  
    public Account(double initialBalance) //parameterized constructor  
    {  
        if(initialBalance >0)  
        {  
            balance = initialBalance;  
        }  
        System.out.println("Parameterized Constructor");  
    }  
}
```

```
    public void credit(double amount)  
    {  
        balance = balance + amount;  
    }  
  
    public void debit(double amount)  
    {  
        if (balance > amount)  
        {  
            balance = balance - amount;  
        }  
    }  
  
    public double getBalance()  
    {  
        return balance;  
    }  
}
```



UML Class
Diagram

AccountTest Class

```
public class Example_Accounts_FloatNumbers {  
  
    public static void main(String[] args) {  
  
        Scanner sc = new Scanner(System.in);  
        Account account1 = new Account(50.00); //balance of account1 = 50.00  
        Account account2 = new Account(); // balance of account2 = 0.00  
  
        System.out.println("Account1 balance = " + account1.getBalance());  
        System.out.println("Account2 balance = " + account2.getBalance());  
  
        double depositAmt;
```

```
System.out.println("Enter deposit amount for account 1");  
depositAmt = sc.nextDouble();  
account1.credit(depositAmt);
```

```
System.out.println("Enter deposit amount for account 2");  
depositAmt = sc.nextDouble();  
account2.credit(depositAmt);
```

```
System.out.println("Account1 balance = " + account1.getBalance());  
System.out.println("Account2 balance = " + account2.getBalance());
```

```
}
```

```
}
```

OUTPUT:

Parameterized Constructor

Default constructor

Account1 balance = 50.0

Account2 balance = 0.0

Enter deposit amount for account 1

500

Enter deposit amount for account 2

250

Account1 balance = 550.0

Account2 balance = 250.0